PART 11–12 • TRUST, SAFETY & POLICY

Security Architecture & Enterprise AI Governance

Tenant isolation, prompt injection defenses, real CVEs, audit logs, and the April 2026 data policy shift

---

TOPICS COVERED

- › Repository & Tenant Isolation
- › Documented Prompt Injection Mitigations
- › Comment & Control Cross-Vendor Attack
- › Content Exclusions & .copilotignore
- › April 2026 Training-Data Policy Change
- › Policy Hierarchy & Conflict Resolution
- › Coding Agent Firewall Architecture
- › Real-World CVE: Agent RCE
- › Data Residency Enforcement
- › Retention by Surface (IDE/CLI/Agent)
- › Enterprise AI Controls (GA Feb 2026)
- › MCP Allowlisting & Custom Agents

PART 11 — Security Architecture

11.1 Repository and Data Access Scope — As Documented by GitHub's Own Trust Center

GitHub's Copilot Trust Center directly addresses how agentic features handle data governance. By default, the Coding Agent's context is limited to the Git contents of the repository where it is working, plus other metadata in that same repository — issues, other pull requests, GitHub Actions logs. The agent accesses data outside that repository (external URLs, APIs) only when explicitly authorized through firewall configuration or by installing an MCP server, since network access is blocked by default. The stated technical safeguard preventing the agent from reaching unauthorized files or data sources is least-privilege scoping on the agent's GitHub credentials combined with the agent firewall.

✓ VERIFIED — Default data-access scope, default-blocked network access, and the least-privilege + firewall safeguard combination, per GitHub's own Copilot Trust Center FAQ

Data accessed by the agent for a specific task is processed within the GitHub Actions runner where that task executes — i.e., processing happens inside the same ephemeral, isolated compute environment that hosts the rest of the agent's work, rather than being shipped to a separate, persistent processing tier.

✓ VERIFIED — Processing location (within the task's own Actions runner), per GitHub Copilot Trust Center

11.2 The Agent Firewall — Architecture and Enterprise Controls

The Coding Agent's firewall is GitHub's primary technical control against both accidental and malicious data exfiltration and prompt injection: by default it restricts the agent's internet access to a GitHub-curated recommended allowlist, and repository or organization administrators can customize this. As of an April 2026 update, organization administrators gained the ability to manage the firewall across every repository in the organization at once — turning the firewall on or off org-wide (or delegating that choice to individual repos), enabling or disabling the recommended allowlist org-wide, adding organization-wide custom allowlist entries (for example, an internal package registry), and controlling whether individual repository admins may add their own custom entries on top.

✓ VERIFIED — Organization-level firewall management capabilities and the April 2026 rollout, per GitHub Changelog 'Organization firewall settings for Copilot cloud agent'

```
# Verified firewall control hierarchy (as of April 2026):

  Organization admin can set, per org:
    - Firewall ON/OFF org-wide, OR delegate to each repo
    - Recommended allowlist ON/OFF org-wide, OR delegate to each repo
    - Org-wide custom allowlist entries (e.g. internal npm/PyPI mirror)
    - Whether repo admins may ADD their own custom entries on top

  Repository admin (if not overridden by org policy) can set, per repo:
    - Firewall ON/OFF for this repository
    - Custom allowlist entries specific to this repository
```

11.3 Documented Prompt Injection Mitigations

GitHub's own documentation enumerates specific, named prompt-injection attack vectors against the Coding Agent and their corresponding mitigations: an attacker may hide instructions inside an issue or comment assigned to Copilot (mitigated by filtering known hidden characters out of user input before it reaches the agent); the agent's internet access is limited by the firewall described above; and access control on who can even trigger the agent matters structurally, since only users with write access to the repository can trigger Copilot by assigning an issue or leaving a comment, and comments from users without write access are never presented to the agent at all.

✓ VERIFIED — Hidden-character filtering, firewall limitation, and write-access-gated triggering, per GitHub Enterprise Cloud Docs 'Using Copilot coding agent'

11.4 The 'Comment and Control' Cross-Vendor Attack — A Real, Documented Bypass

In April 2026, security researcher Aonan Guan publicly disclosed an attack dubbed "Comment and Control," confirmed to work against three separate widely-used AI coding agents: Anthropic's Claude Code Security Review, Google's Gemini CLI Action, and GitHub's Copilot Agent (referred to in the disclosure as the SWE Agent). The attack is proactive rather than reactive: because GitHub Actions workflows auto-trigger on events like pull_request, issues, and issue_comment, simply opening a PR or filing an issue with a crafted payload can activate the agent without any further victim action — except in Copilot's specific case, where the attacker's issue must still be manually assigned to Copilot by the victim, a meaningful structural difference from the other two tools.

✓ VERIFIED — Comment and Control attack, the three confirmed-vulnerable tools, and the manual-assignment caveat specific to Copilot, per SecurityWeek's coverage of the April 2026 disclosure

The disclosure documented that GitHub's Copilot Agent finding involved bypassing three separate, independently implemented runtime-level mitigations GitHub had built specifically to prevent this class of attack: environment-variable filtering, secret scanning, and the network firewall. According to the published technical analysis, environment-variable filtering only stripped sensitive variables from the immediate child bash subprocess, but running ps auxeww from within that same subprocess could read /proc/[pid]/environ for the unfiltered parent Node.js process and an associated MCP server, recovering the GITHUB_TOKEN, a GITHUB_COPILOT_API_TOKEN, and two additional credentials. GitHub's secret scanner was reported to detect known token prefixes such as ghs_ and ghu_, but base64-encoding the recovered credentials before committing them was sufficient to evade that pattern matching. Finally, the encoded credential file was exfiltrated via an ordinary git push to a pull request — a whitelisted, completely normal Copilot workflow operation — making the exfiltration indistinguishable from routine agent activity to the firewall.

✓ VERIFIED — The specific three-stage bypass (env-var filtering bypass via /proc/[pid]/environ, secret-scanner bypass via base64 encoding, firewall bypass via whitelisted git push) per CyberSecurityNews' detailed technical writeup of the April 2026 disclosure

■ *This is not a hypothetical or theoretical vulnerability discussion: this is a documented, publicly disclosed, vendor-confirmed bypass of three independent, purpose-built security mitigations in a system this report has otherwise described as having a credible security architecture. The lesson for any reader designing a similar enterprise agent platform is that defense-in-depth controls implemented within the SAME execution runtime as the secrets they protect can all be defeated by a single privilege-escalation primitive (here, process-environment introspection) if the agent has shell access in that runtime.*

The disclosure's stated implication, attributed to the researcher, generalizes well beyond GitHub Actions specifically: the underlying pattern — untrusted input flowing into an AI agent that holds production secrets and unrestricted tool access within the same runtime — applies to any agent that processes untrusted input with access to tools and secrets, including Slack bots, Jira agents, email agents, and deployment automation, not solely

CI/CD-triggered coding agents.

11.5 A Separate, Earlier Disclosure: Remote Code Execution via the IDE Agent

Independently, security researcher Johann Rehberger (Embrace The Red) reported a separate vulnerability (assigned CVE-2025-53773) in which a prompt injection — placed directly inside a source code file the developer opens — could cause VS Code's GitHub Copilot agent mode to modify the workspace's own settings.json file to enable "YOLO mode" (auto-approval of actions without confirmation), at which point the agent could execute arbitrary terminal commands, demonstrated in the disclosure by popping a calculator and, more seriously, described as sufficient to join the developer's machine to a botnet. The researcher additionally noted that the AI's ability to write to its own configuration and permission files — not just application code — is a recurring, broader vulnerability pattern: an agent that can modify the settings governing what it itself is allowed to do, including files like .vscode/tasks.json or allow-listed bash commands for other co-located agents, can self-escalate its own privileges.

✓ VERIFIED — CVE-2025-53773 and its mechanics (prompt injection → settings.json modification → YOLO mode → arbitrary command execution), per Embrace The Red's published disclosure, including documented coordination with Microsoft after the June 29, 2025 report

11.6 Defense-in-Depth Recommendations from Third-Party Security Practice

Independent security tooling vendors building on top of GitHub's native protections argue that the firewall, while a critical baseline, provides no visibility into what processes actually ran, which APIs were actually hit, or what packages were actually installed during an agent session — meaning a security team troubleshooting after an incident cannot answer those questions from native GitHub Actions logs alone, and recommend runtime-level monitoring layered on top of (not instead of) GitHub's firewall.

■ INFERRED — This specific recommendation (layer third-party runtime monitoring on top of GitHub's firewall) is a security vendor's commercial argument as well as a technical observation; it is included because the underlying technical gap it identifies — lack of native process/API-level visibility — is independently plausible and consistent with the firewall's documented scope (network-level allowlisting, not full runtime introspection).

• Use least-privilege secrets: agents performing read-only tasks (e.g., issue triage) should never hold a token with write scope.

• Require human approval gates before an agent performs outbound actions or accesses credentials, rather than relying solely on automated filters.

• Audit all AI agent integrations in CI/CD pipelines specifically, and monitor Actions logs for anomalous credential-access patterns.

✓ VERIFIED — These three specific recommendations are attributed directly to the Comment and Control disclosure's own published guidance, per CyberSecurityNews' coverage

PART 12 — Enterprise AI Governance

12.1 Enterprise AI Controls and the Agent Control Plane (GA February 2026)

GitHub announced general availability of a consolidated "Enterprise AI Controls" surface and an "agent control plane" in February 2026, explicitly designed to give enterprise administrators deeper control and greater auditability over AI and agent usage across their environments. AI Controls in enterprise settings functions as a single consolidated navigation point for AI-related administrative tasks, and this administrative responsibility can be decentralized to dedicated AI-standards teams via an enterprise custom role with fine-grained permissions to view audit logs, agent session activity, and manage AI Controls — without granting those teams broader enterprise-owner privileges.

✓ VERIFIED — GA date, consolidated AI Controls navigation, and the dedicated enterprise custom role for AI governance, per GitHub Changelog (Feb 26, 2026)

At GA, the feature set included viewing cloud-agent session activity from the prior 24 hours, managing an enterprise-wide MCP allowlist through a centralized MCP registry URL (this specific sub-feature remained in preview even as the broader AI Controls GA shipped), searching agentic session activity filtered by specific agents including third-party agents, and tracking usage by organization within the enterprise. A dedicated, version-controllable custom-agent definition path (.github/agents/*.md) can be protected enterprise-wide via a one-click push rule, preventing unauthorized edits to the agent definitions that encode organizational standards.

✓ VERIFIED — All feature details in this paragraph, including the MCP-allowlist preview-status carve-out and the .github/agents/*.md push-rule protection mechanism, per GitHub Changelog (Feb 26, 2026)

12.2 Policy Hierarchy and Conflict Resolution — A Documented, Non-Obvious Mechanism

GitHub's policy documentation specifies precise, non-default-intuitive conflict-resolution rules for how Copilot policies combine across multiple organizations and enterprises. If a single user receives a Copilot license from two organizations within the same enterprise that have configured the same policy differently, the LEAST restrictive policy usually applies (with documented exceptions). If a user instead receives licenses from two different enterprises entirely, the MOST restrictive policy across those enterprises almost always applies — GitHub's own example: if any one enterprise disables Copilot Chat in GitHub.com, that feature is disabled for the user, full stop, regardless of what any other enterprise granting them a license has configured.

✓ VERIFIED — The least-restrictive-within-enterprise / most-restrictive-across-enterprises conflict resolution rules, and the specific Copilot Chat example, per GitHub Docs 'GitHub Copilot policies for enterprises and organizations'

■ *This is a critical, easy-to-miss governance detail for any organization operating multiple GitHub enterprises (e.g., post-acquisition, or with separate enterprises for regulated vs. unregulated business units): a restrictive AI policy set in ANY one enterprise a user belongs to silently overrides a more permissive policy in another, with no UI warning at the point the more permissive enterprise's admin sets their own policy.*

12.3 Audit Logs — Scope and a Documented Limitation

GitHub's Copilot-specific audit log records changes to Copilot plan settings and policies, and license assignment/removal events, searchable via the action:copilot query and filterable to agent-specific activity via actor:Copilot. Audit log retention is 180 days by default, with GitHub explicitly recommending streaming to a SIEM platform for longer-term history and anomaly alerting. Critically, GitHub's own documentation states a clear limitation: the audit log does NOT include client session data, such as the actual prompts a user sends to Copilot locally — accessing that level of detail requires a custom solution, and GitHub notes that some companies use custom hooks to send Copilot CLI events to their own external logging service to fill this gap.

✓ VERIFIED — 180-day retention, SIEM-streaming recommendation, and the explicit 'audit log does not include client session prompts' limitation (plus the custom-hook workaround), per GitHub Docs 'Reviewing audit logs for GitHub Copilot'

■ *This is an important, non-obvious gap for compliance teams: an enterprise audit log proving Copilot was enabled for a user, and even that an agent session occurred, is not the same as a log of what that user or agent actually asked the model — the latter requires deliberate additional engineering on the customer's side.*

12.4 Repository and Model Allowlisting

Organization-level Copilot settings include a dedicated "Models" policy page, distinct from general feature policies, specifically governing the availability of models beyond the basic models bundled with a Copilot plan — explicitly flagged in GitHub's own documentation as potentially incurring additional cost, meaning model-level governance is also a cost-control lever, not purely a security one. Separately, organizations can restrict which AI models members may access at the Copilot CLI level specifically, with the CLI's model picker and fallback behavior (including an optional automatic fallback to an "auto" model when the primary model is rate-limited) governed by these same organizational policies.

✓ VERIFIED — Dedicated org-level Models policy page and its cost implication, per GitHub Docs 'Managing policies and features for GitHub Copilot in your organization'; CLI-specific model restriction and rate-limit fallback behavior, per GitHub's copilot-cli technical documentation

12.5 The April 2026 Training-Data Policy Change — A Genuinely Contested, Recent Shift

This is the single area in this entire report where GitHub's own stated position, independent journalism, and the affected developer community are in the most active, visible tension, and where the underlying facts changed very recently relative to this report's writing. GitHub itself, via an official Privacy Statement and Terms of Service update plus a dedicated company blog post and a public FAQ thread on GitHub's own community forum, confirmed that as of April 24, 2026, interaction data (prompts, suggestions, outputs) from Free, Pro, and Pro+ individual-tier Copilot users may be used to train and improve GitHub's models on an opt-out basis — meaning the setting defaults to ON for those tiers, and a user must actively find and use an opt-out control to prevent it.

■ CONTESTED / RECENT — GitHub's own community FAQ thread directly confirms the April 24, 2026 effective date, the opt-out (not opt-in) default for Free/Pro/Pro+ tiers, and that GitHub and Microsoft personnel working on AI model development may access this collected interaction data, along with service providers under contractual restriction. This is GitHub's own first-party statement, but the policy is new enough, and significant enough, that this report flags it as CONTESTED/RECENT rather than simply VERIFIED-and-settled: the practical opt-out mechanics, the precise scope of 'interaction data,' and how this interacts with previously stated 'Copilot doesn't train on your code' marketing are all live points of community concern at the time of writing, and a reader evaluating this for a real deployment should re-check GitHub's current live policy page directly rather than relying on this snapshot.

GitHub's stated rationale, drawn directly from its own FAQ, is that as Copilot usage continues to increase dramatically, the company has identified a need for real-world data to help its models cover the growing number of scenarios they are now used for, and that the company is committed to giving developers control over whether their interaction data is used for training and will always be transparent about that use.

Critically, and consistently across every source reviewed for this report — GitHub's own FAQ, the official blog post, and independent third-party reporting — Copilot Business and Copilot Enterprise customers are explicitly and contractually excluded from this change: GitHub states directly that its agreements with Business and Enterprise customers prohibit using their Copilot interaction data for model training, and that GitHub honors those commitments. This Business/Enterprise exclusion is the single most important fact for any enterprise reader of this report to retain from this section.

✓ VERIFIED — The Business/Enterprise contractual exclusion from the April 2026 training-data change is independently confirmed by GitHub's own community FAQ post, GitHub's official blog post on the policy update, and third-party technical commentary (Windows Forum, DEV Community) — this specific exclusion is the most solidly corroborated claim in this entire section, even though the broader policy shift it sits within is appropriately flagged as recent/contested.

Independent commentary has been notably more skeptical in framing than GitHub's own announcement. One April 2026 community-sourced privacy/security tracker characterizes the change directly as meaning the prior assumption "I pay for Pro so my code isn't training anything" stopped being categorically true on that date unless a user actively opts out, and separately notes that retention practices are not uniform across surfaces even setting training aside: IDE code completions are processed without retaining the prompt/suggestion by default, while "outside the IDE" surfaces such as the CLI and agent-style workflows fall under a separate bucket where input/output may be retained for roughly 28 days for abuse/troubleshooting purposes before deletion — and Coding Agent session logs specifically are retained for the life of the account in order to provide the service, a materially longer retention period than the 28-day CLI/agent bucket might suggest in isolation.

■ CONTESTED / RECENT — The 28-day retention figure for 'outside the IDE' surfaces and the Coding Agent's account-lifetime session-log retention are both drawn from a mix of GitHub's own Trust Center FAQ and independent community synthesis; treat the precise day-counts as indicative of GitHub's general posture (longer retention for agentic/CLI surfaces than for in-IDE completions) rather than as a guaranteed-stable number, since retention policy is exactly the kind of detail that tends to change alongside the broader data-use policy shifts described above.

12.6 Data Retention Summary Table (Business/Enterprise, as separately confirmed)

Surface	Prompts/Suggestions retained?	User engagement data	Notably excluded from tra
IDE (Chat, Completions, CLI per Trust Center)	Not retained by default	Kept 2 years	Yes — Business/Enterprise contractually excl
'Outside the IDE' surfaces (per community synthesis)	~28 days for abuse/troubleshooting	Kept 2 years	Yes — Business/Enterprise contractually excl
Coding Agent session logs	Retained for life of account (service provision)	Kept 2 years	Yes — Business/Enterprise contractually excl
Free / Pro / Pro+ (individual) interaction data	Used for training unless opted out (as of Apr 24, 2026)	Kept 2 years	No — opt-out, not excluded, by default

■ *The 'Not retained by default' and '2 years' figures for the top three rows are drawn directly from GitHub's own Copilot Trust Center FAQ; GitHub's own FAQ further notes GitHub may retain input/output data for a limited, targeted, time-limited period specifically to investigate confirmed Acceptable Use Policy or Terms of Service violations, or to protect the security and integrity of its services — an exception to the 'not retained' default that exists across tiers.*

12.7 Compliance Reporting and Data Residency Recap

As established in Part 10.2, GitHub Enterprise Cloud with data residency provides a token-scoped, routing-level enforcement mechanism (not merely a policy promise) restricting Copilot inference to model endpoints within a designated region, currently available for the EU, Australia, the US, and Japan. Separately, GitHub's Copilot usage and code-generation metrics dashboards are now available, including via API, to GitHub Enterprise Cloud with data residency customers specifically — a deliberate extension of observability tooling to the data-residency product tier, recognizing that compliance-focused customers need the same usage visibility as standard GHEC customers.

✓ VERIFIED — Data residency dashboard/API extension, per GitHub Changelog (Jan 29, 2026)

Key Takeaways — Parts 11–12

• GitHub's documented security architecture for the Coding Agent (single-repo scope, default-blocked network access, least-privilege credentials, write-access-gated triggering, hidden-character filtering) is real and verifiable — but is not unbreakable: the April 2026 Comment and Control disclosure is a documented, vendor-confirmed bypass of three independent runtime mitigations, using process-environment introspection, encoding-based filter evasion, and abuse of a whitelisted git push as the exfiltration channel.

• The core lesson generalizing beyond GitHub specifically: co-locating an agent's secrets and its shell/tool access within the same execution runtime as the untrusted input it processes is a structural risk that firewalls and pattern-matching secret scanners alone do not close.

• Enterprise AI Controls (GA Feb 2026) gives GitHub enterprises a genuinely consolidated governance surface, but its policy-conflict-resolution rules (least-restrictive within an enterprise, most-restrictive across enterprises) are non-obvious and merit explicit documentation in any organization spanning multiple GitHub enterprises.

• The native Copilot audit log does not capture actual prompt content — only plan/policy/license events — which is a hard limit compliance teams must engineer around themselves.

• The April 2026 individual-tier training-data policy change is real, GitHub-confirmed, and the Business/Enterprise contractual exclusion from it is the best-corroborated fact in this section — but the policy is recent enough, and contested enough in community reception, that any reader relying on this report for a live compliance decision should re-verify directly against GitHub's current published policy rather than treating this snapshot as permanently current.